

React JS

Title: React JS - Beginner to Advanced Notes

By: VK AI TECH by Kabir

1. What is React JS?

React JS is an **open-source JavaScript Library** developed by **Meta (Facebook)** for building **fast, interactive, and reusable User Interfaces (UI)**.

It follows a **Component-Based Architecture**, where every part of the UI is divided into reusable components.

2. Why do we need React JS for creating UI?

Traditional HTML + JavaScript becomes difficult to create and maintain UI as applications grow.

Examples

- Facebook
- Instagram
- Netflix
- WhatsApp Web
- Gmail

All these applications constantly update their UI.

React makes these updates:

- Faster
- Cleaner
- Easier
- Reusable

Advantages

- ✓ Reusable Components
 - ✓ Faster Rendering
 - ✓ Better Performance
 - ✓ Easy State Management
 - ✓ Large Community
 - ✓ Virtual DOM
 - ✓ Easy Maintenance
-

3. Drawbacks of HTML and JavaScript to Create UI

Drawbacks of HTML

1. Cannot Create Dynamic UI

- HTML is a **static markup language**.
- It displays fixed content unless JavaScript updates it.
- Changing data requires manual DOM manipulation.

Example:

```
<h1>Welcome Kabir</h1>
```

To change **Kabir** to **Rahul**, HTML alone cannot do it.

2. Cannot Reuse Code

- HTML does not support reusable components.
- If the same UI is needed multiple times, we must copy and paste the code.
- Updating one section requires updating every copied section manually.

Example:

```
<div class="card">Product 1</div>  
<div class="card">Product 2</div>  
<div class="card">Product 3</div>
```

The same structure is repeated again and again.

Drawbacks of JavaScript

1. Complex to Create UI

- Creating UI using JavaScript requires writing many lines of code.
- Every HTML element must be created manually.

Example:

```
const h1 = document.createElement("h1");  
h1.innerText = "Hello";  
document.body.appendChild(h1);
```

2. Need to Deal with Complex DOM Manipulation

- Developers must manually select, create, update, and remove DOM elements.
- As applications grow, DOM manipulation becomes difficult to manage.
- More code increases the chances of bugs.

Example:

```
const heading = document.getElementById("title");
heading.innerText = "React JS";
heading.style.color = "blue";
```

4. What is React Element?

A React Element is a **JavaScript Object** that represents UI elements on the Web Page.

React later converts it into DOM elements.

Example

```
const element =
  React.createElement(
    "h1",
    {},
    "Hello"
  );
```

React Element internally becomes

```
{
  type: "h1",
  props: {},
  children: "Hello"
}
```

5. How to create React Element

A. Single Element

```
const heading =  
React.createElement(  
  "h1",  
  {},  
  "Welcome"  
);
```

B. Multiple Elements

```
const parent =  
React.createElement(  
  "div",  
  {},  
  [  
    React.createElement("h1", {}, "Heading"),  
    React.createElement("p", {}, "Paragraph")  
  ]  
);
```

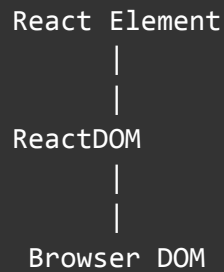
Hierarchy

```
div  
|  
|--h1  
|  
|--p
```

6. What is ReactDOM?

ReactDOM is responsible for rendering React Elements into the Browser DOM.

Think of it as a bridge.



Example

```
const root =  
ReactDOM.createRoot(  
document.getElementById("root")  
);  
  
root.render(element);
```

7. Why do we use JSX instead of React.createElement()?

Creating UI using React.createElement()

```
React.createElement(  
"div",  
{},  
React.createElement(  
"h1",  
{},
```

```
"Hello"  
)  
)
```

Looks difficult to write or implement.

Now, Have a Look on JSX

```
<div>  
  <h1>Hello</h1>  
</div>
```

Advantages

- ✓ Easy to Read
 - ✓ Easy to Write
 - ✓ Looks like HTML
 - ✓ Less Code
 - ✓ Better Maintenance
-

8. What is JSX?

- JSX means JavaScript XML
- It allows us to write HTML like syntax inside JavaScript.
- A browser cannot understand JSX.
- It must be converted into JavaScript.
- Babel convert

```
<h1>Hello </h1>
```



```
React.createElement(  
  "h1",  
  {},  
  "Hello"  
)
```

JSX Rules 🙌

Rule 1

Return only one parent element



```
<div>  
  
</div>
```



```
<h1></h1>  
<p></p>
```


Rule 2

```
Close every tag  
<img />  
<input />
```

Rule 3

```
Use camelCase  
onClick  
  
className  
  
htmlFor
```

Rule 4

JavaScript inside {}

```
<h1>{name}</h1>
```

9. How to create JSX Elements

Single Element

```
const element = <h1>Hello React</h1>;
```

Nested Elements

```
const element = (  
  <div>  
    <h1>Hello</h1>  
    <p>React</p>  
  </div>
```

)

10. Difference between HTML and JSX

Difference Between HTML and JSX

HTML

HTML is a markup language used to create web pages.

Uses `class` attribute.

Uses `for` attribute.

Uses lowercase event names like `onclick`.

Uses `tabindex` attribute.

Uses `readonly` attribute.

JSX

JSX is a JavaScript syntax used to create UI in React.

Uses `className` attribute.

Uses `htmlFor` attribute.

Uses camelCase event names like `onClick`.

Uses `tabIndex` attribute.

Uses `readOnly` attribute.

Uses `maxLength` attribute.

Uses `maxLength` attribute.

Boolean attributes can be written without a value (`checked`).

Boolean attributes are passed as JavaScript values (`checked={true}`).

Inline CSS is written as a string (`style="color:red"`).

Inline CSS is written as a JavaScript object (`style={{color: "red" }}`).

Cannot directly use JavaScript expressions.

Can use JavaScript expressions inside `{}`.

The browser understands HTML directly.

JSX must be converted into JavaScript using Babel before rendering.

Multiple root elements are allowed in an HTML document.

A component must return a single parent (root) element.

Some tags don't require self-closing.

Every tag must be properly closed (e.g., `<img />`).

Mainly used for static web page structure.

Mainly used for building dynamic and interactive React applications.

11. Vite Build Tool

Vite is a modern build tool used to create React applications.

Why Vite?

- ✓ Very Fast
 - ✓ Hot Module Replacement
 - ✓ Faster Development Server
 - ✓ Optimized Production Build
 - ✓ Supports TypeScript
 - ✓ Less Configuration
-

12. How to create React App?

```
npm create vite@latest
```

```
Project Name  
  react-app
```

```
Select  
  React
```

```
Select  
  JavaScript
```

```
Go inside  
  cd react-app
```

```
Install packages  
  npm install
```

```
Run project  
  npm run dev
```

Why Build Tool?

A Build Tool

- Bundles files
 - Optimizes code
 - Converts JSX
 - Runs Development Server
 - Supports Modules
 - Handles Images
 - Hot Reload
-

13. Problems without Build Tool

Without Vite, Need to manually

- Import React CDN
- Import ReactDOM CDN
- Import Babel
- Configure Modules
- Configure Bundling
- Configure Minification

Problems

- ✗ Slow
 - ✗ Manual Configuration
 - ✗ No Hot Reload
 - ✗ Difficult Project Structure
-

14. React Folder Structure

```
project/  
  node_modules/  
  public/  
  src/  
    assets/  
    App.jsx  
    App.css  
    main.jsx  
    index.css  
  
package.json  
vite.config.js  
README.md
```

What is React JS Project Structure?

- React JS Project Structure is the organized arrangement of files and folders in a React application.
 - It helps developers write clean, maintainable, and scalable code.
 - Every folder has a specific responsibility.
-

Why Do We Need JS Project Structure?

- To keep the project organized.
 - To easily find files.
 - To separate different responsibilities.
 - To make collaboration easier in a team.
 - To maintain large applications efficiently.
-

Purpose of Project Structure

- Organize application code.
 - Improve readability.
 - Reduce code duplication.
 - Make debugging easier.
 - Support scalability as the application grows.
 - Help developers follow a standard folder organization.
-

Developer's Role

A React Developer should:

- Create a proper folder structure.
 - Keep reusable components in separate folders.
 - Separate UI, logic, styles, and assets.
 - Write clean and maintainable code.
 - Follow naming conventions.
 - Make the project easy for other developers to understand.
-

Basic Vite Project Structure

```
my-react-app/  
├── node_modules/  
├── public/  
├── src/  
│   ├── assets/  
│   ├── components/  
│   ├── App.jsx  
│   ├── main.jsx  
│   └── index.css  
├── .gitignore  
├── eslint.config.js  
├── index.html  
├── package.json  
├── package-lock.json  
└── vite.config.js
```

Folder & File Explanation

1. node_modules/

What?

Contains all installed npm packages.

Purpose

Stores project dependencies.

Developer Role

- Never modify manually.
- Generated automatically after `npm install`.

2. public/

What?

Stores static files.

Purpose

Files are served directly without processing.

Examples

- favicon.ico
- images
- robots.txt

Developer Role

Keep static assets that do not need bundling.

3. src/

What?

Main folder of the React application.

Purpose

Contains all application source code.

Developer Role

Write most of the project code inside this folder.

4. assets/

What?

Stores project resources.

Purpose

Manage images, icons, fonts, videos, etc.

Developer Role

Keep all application assets organized.

5. App.jsx

What?

Main application component.

Purpose

Combines different components to build the application UI.

Developer Role

Manage the application's overall layout and render child components.

7. main.jsx

What?

Entry point of the React application.

Purpose

Starts the React application and renders the root component into the DOM.

Developer Role

Initialize React and connect the app to `index.html`.

8. index.css

What?

Global stylesheet.

Purpose

Apply styles across the application.

Developer Role

Write common CSS used throughout the project.

9. index.html

What?

The only HTML file in a Vite React project.

Purpose

Provides the root element where the React app is mounted.

Developer Role

Usually contains:

```
<div id="root"></div>
```

React renders the entire application inside this div.

10. package.json

What?

Project configuration file.

Purpose

Stores:

- Project name
- Dependencies
- Scripts
- Project metadata

Developer Role

Install packages and manage project scripts.

11. package-lock.json

What?

Dependency lock file.

Purpose

Locks the exact versions of installed packages.

Developer Role

Commit this file to version control for consistent installations.

12. vite.config.js

What?

Vite configuration file.

Purpose

Customize Vite's behavior, plugins, aliases, and build settings.

Developer Role

Configure the project when customization is needed.

13. eslint.config.js

What?

ESLint configuration file.

Purpose

Defines coding rules and helps maintain code quality.

Developer Role

Follow consistent coding standards and reduce errors.

14. .gitignore

What?

Git ignore configuration.

Purpose

Specifies files and folders Git should not track.

Common Entries

- node_modules/
- dist/
- .env

Developer Role

Prevent unnecessary or sensitive files from being committed.

15. README.md

What?

Project documentation file.

Purpose

Provides information about the project.

Includes

- Project overview
- Installation steps
- Commands
- Features
- Usage instructions

Developer Role

Maintain clear documentation for the project.

15. Props

1. Props means Properties
2. Props are used to send data

Example

```
<Greeting  
  name="Kabir"  
>
```

```
function Greeting(props){  
  
  return(  
    <h1>  
      {props.name}  
    </h1>  
  )  
  
}
```

Props are

- ✓ Read Only
 - ✓ Immutable
-

16. Functional Component

A Functional Component is a JavaScript Function that returns JSX.

Example

```
function Header(){  
  
  return(  
  
    <h1>  
    Welcome  
    </h1>  
  
  );  
  
}
```

Rules

- ✓ First Letter Capital
- ✓ Return JSX
- ✓ Reusable

React JS – State

What is State?

👉 **State is a special variable that stores data inside a component.**

Whenever state changes, React automatically updates the UI.

Why do we need State?

Without state, changing a variable does **not** update the UI.

State tells React:

"The data has changed. Please render the component again."

Real Life Example

Think about a mobile phone.

Battery : 100%

↓

Battery : 90%

↓

Battery : 70%

Battery percentage keeps changing.

This changing value is called **State**.

Another examples:

- Cart Items

- Login Status
- Theme (Dark/Light)
- Counter Value
- User Information

All these values change with time.

What is useState?

useState is a React Hook used to create State inside a Functional Component.

It allows React to remember values between renders.

Syntax

```
const [stateVariable, setStateFunction] = useState(initialValue);
```

Breakdown

```
const [count, setCount] = useState(0);
```

count

→ Current value

setCount

→ Function used to update state

0

→ Initial value

Diagram

`useState(0)`

↓

`count = 0`

↓

`setCount()`

↓

count changes

↓

React Re-renders Component

↓

Updated UI

Rules

- ✓ `useState` must be inside Functional Component
 - ✓ `useState` should be called at Top Level
 - ✓ Never call inside
 - `if`
 - `for`
 - `while`
 - `switch`
-

Example 1 (Counter)

```
import { useState } from "react";

function App(){

  const [count, setCount] = useState(0);

  return(
    <>
    <h1>{count}</h1>

    <button
      onClick={() => setCount(count + 1)}
    >
      Increase
    </button>

    </>
  )
}
```

Initially

```
count = 0
```

Screen

0

Click Button

```
setCount(1)
```

React stores

```
count = 1
```

Component renders again

Screen

1

Again Click
setCount(2)

Screen

2

Again

3

Again

4

Flow

Button Click

↓

setCount()

↓

State Updated

↓

React Re-render

↓

Updated UI

Example 2 (Light ON / OFF)

```
import { useState } from "react";

function App(){

  const [isOn, setIsOn] = useState(false);

  return(
    <>
    <h2>

    {isOn ? "💡 Light ON" : "🌙 Light OFF"}

    </h2>

    <button
    onClick={() => setIsOn(!isOn)}
    >

    Toggle

    </button>

    </>
    )
  }
```

Initially

false

Screen

Light OFF

Click

true

Screen

Light ON

Click Again

false

Screen

Light OFF

What is setState?

setState is the function returned by **useState** that updates the state value.

Whenever we call `setState`,

React

1. Updates the state
2. Re-renders the component
3. Updates the UI

Syntax

```
setState(newValue);
```

Example

```
setCount(10);
```

React changes
count

0

↓

10

Component renders again.

Two Common useState Use Cases

Use Case 1 – Counter

Used in

- Like Button
- Quantity Increase
- Rating
- Cart Quantity

```
const [count, setCount] = useState(0);

<button
  onClick={() => setCount(count + 1)}
  >
    Increase
  </button>
```

Use Case 2 – Show / Hide Password

```
const [showPassword, setShowPassword] = useState(false);

<input
  type={showPassword ? "text" : "password"}
  />

<button
  onClick={() => setShowPassword(!showPassword)}
```

```
>  
  
Show / Hide
```

```
</button>
```

```
Initially  
false
```

```
Password hidden  
↓  
Click  
true
```

```
Password visible  
↓  
Click Again  
false
```

```
Password hidden
```

Difference Between Variable and State

Variable	State
Stores value	Stores value
Changing value does not update UI	Changing value updates UI
React does not track it	React tracks it
No re-render	Re-render happens

Interview Questions

1. What is State?

State is a built-in object that stores dynamic data in a component and updates the UI whenever its value changes.

2. What is useState?

useState is a React Hook that creates and manages state in Functional Components.

3. What does setState do?

It updates the state value and tells React to re-render the component.

4. Why don't normal variables update the UI?

Because React only tracks **state**, not ordinary variables.

5. When should we use State?

Use state whenever a value needs to change over time and the UI should reflect that change, such as counters, forms, toggles, user input, cart items, login status, or fetched data.

React Router 🤔

What is React Router?

- React Router is a **library** used for navigation in React applications.
- It allows users to move between different pages **without reloading the browser**.
- It enables **Single Page Application (SPA)** navigation.

Why Do We Need React Router?

- To navigate between multiple pages.
 - To avoid full page reloads.
 - To create Single Page Applications (SPA).
 - To improve user experience.
 - To manage application routes efficiently.
-

Purpose of React Router

- Client-side routing.
 - Faster page navigation.
 - Better user experience.
 - Organize application pages.
 - Support nested routing and layouts.
-

Developer's Role

A React Developer should:

- Define application routes.
 - Create reusable layouts.
 - Handle navigation.
 - Protect private routes.
 - Organize nested routes properly.
-

React Router Installation

npm install react-router-dom

BrowserRouter

What is BrowserRouter?

- BrowserRouter is the **root router component**.
 - It enables routing using the browser's History API.
 - It should wrap the entire React application.
-

Why do we use BrowserRouter?

- To enable routing.
 - To manage browser URLs.
 - To support forward/back navigation.
-

Example

```
import { BrowserRouter } from "react-router-dom";  
  
<BrowserRouter>  
  <App />  
</BrowserRouter>
```

Routes

What is Routes?

- Routes is a container for multiple Route components.
 - It checks the current URL.
 - It renders the matching Route.
-

Purpose

- Groups all routes.
 - Matches only one route at a time.
-

Example

```
<Routes>  
  
</Routes>
```

Route

What is Route?

- Route connects a **URL path** with a React component.
-

Purpose

- Display a specific component for a URL.
-

Example

```
<Route path="/" element={<Home />} />

<Route path="/about" element={<About />} />

<Route path="/contact" element={<Contact />} />
```

Link

What is Link?

- Link is used for navigation.
 - It changes pages **without refreshing** the browser.
-

Why use Link?

- Faster navigation.
 - Prevents page reload.
 - Maintains SPA behavior.
-

Example

```
<Link to="/">Home</Link>

<Link to="/about">About</Link>

<Link to="/contact">Contact</Link>
```

Navigate

What is Navigate?

- Navigate redirects users to another page programmatically.
 - It is commonly used after login, logout, or access checks.
-

Purpose

- Redirect users.
 - Handle authentication flow.
 - Protect routes.
-

Example

```
<Navigate to="/login" />
```

Outlet

What is Outlet?

- Outlet is a placeholder where child routes are displayed.
 - It is mainly used in Nested Routing.
-

Why use Outlet?

- Share common layouts.
 - Avoid repeating Navbar, Sidebar, Footer.
 - Render child pages inside a parent layout.
-

Example

```
function Dashboard(){  
  return(  
    <>  
      <Navbar />  
  
      <Outlet />  
  
      <Footer />  
    </>  
  )  
}
```

Nested Routing

What is Nested Routing?

- Nested Routing means creating **child routes inside a parent route**.
 - Child pages are rendered inside the parent's `<Outlet />`.
-

Why use Nested Routing?

- Reuse layouts.
 - Keep routing organized.
 - Build dashboard-style applications.
 - Avoid duplicate code.
-

Parent Route

```
<Routes>
```

```
<Route path="/dashboard" element={<Dashboard />}>

  <Route path="profile" element={<Profile />} />

  <Route path="settings" element={<Settings />} />

</Route>

</Routes>
```

Parent Component

```
function Dashboard(){

  return(
    <>
      <Navbar />

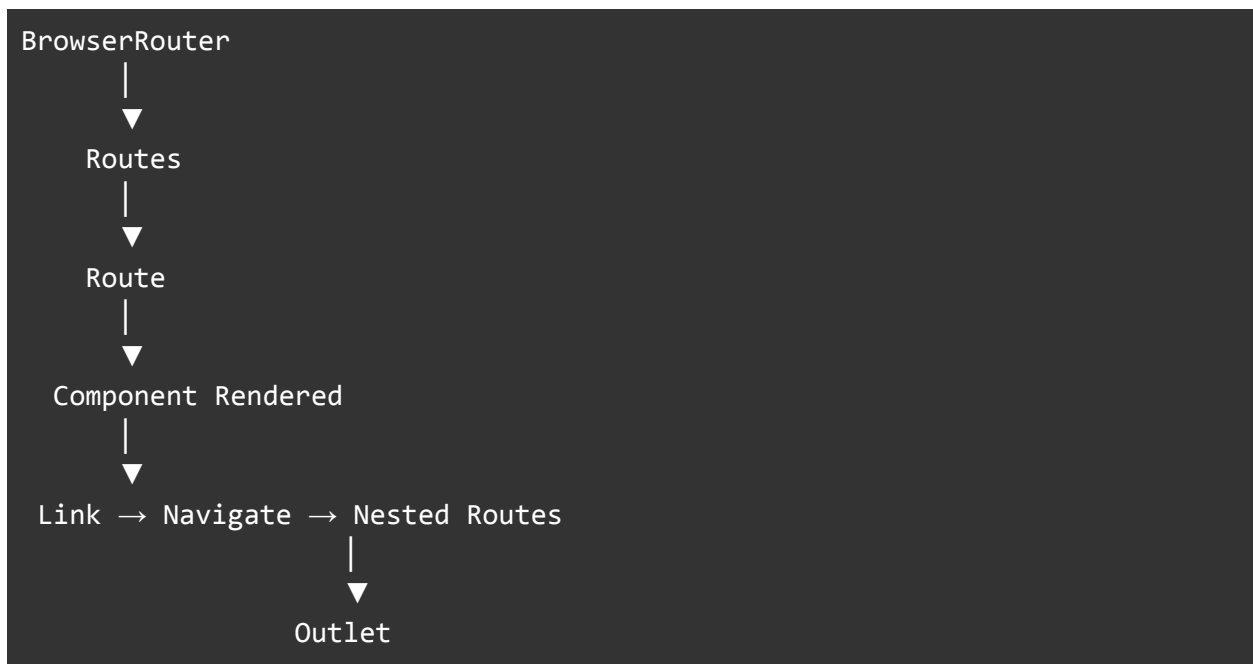
      <Outlet />

      <Footer />
    </>
  )
}
```

URL Structure

```
/dashboard
/dashboard/profile
/dashboard/settings
```

Routing Flow



Props Drilling in React JS

What is Props Drilling?

- Props Drilling is the process of **passing data from a parent component to a deeply nested child component through intermediate components**.
- The intermediate components receive the props even if they don't use them.

Why Do We Need Props Drilling?

- React follows **one-way data flow**.
- A parent component passes data to its child using props.
- If a deeply nested component needs the data, it must be passed through every intermediate component.

Purpose of Props Drilling

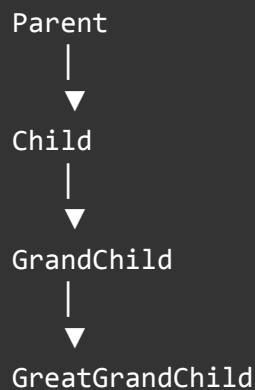
- Share data between parent and child components.
 - Maintain one-way data flow.
 - Pass state or values to nested components.
-

Developer's Role

A React Developer should:

- Use props drilling only when a few components are involved.
 - Avoid excessive props drilling in large applications.
 - Use Context API, Redux Toolkit, or Zustand when many nested components need the same data.
 - Keep components simple and reusable.
-

Example With Props Drilling



The `name` prop is passed through every component until it reaches the component that needs it.

```
function Parent() {
  const name = "Kabir";

  return <Child name={name} />;
}

function Child({ name }) {
  return <GrandChild name={name} />;
}

function GrandChild({ name }) {
  return <GreatGrandChild name={name} />;
}

function GreatGrandChild({ name }) {
  return <h1>{name}</h1>;
}
```

Problem with Props Drilling

- Too many props are passed through components.
 - Intermediate components may not use the data.
 - Code becomes difficult to read and maintain.
 - Refactoring becomes harder as the application grows.
-

Solution to Props Drilling

Instead of passing props through multiple levels, use state management solutions such as:

- Context API

These allow components to access shared data directly without unnecessary prop passing.

Real-World Example



Suppose the user's name is stored in **App**, but only **ProfileCard** needs it. With props drilling, the **name** prop must be passed through **Dashboard** and **Profile** even though they don't use it.

Advantages

- Simple for small applications.
 - Easy to understand.
 - Follows React's one-way data flow.
 - No additional libraries required.
-

Disadvantages

- Too much prop passing.
- Unnecessary code in intermediate components.
- Hard to maintain in large applications.
- Reduces code readability.
- Difficult to scale.

● Context API in React JS

● 1. What is Context API?

The **Context API** is a built-in feature of React that allows us to **share data between multiple components without passing props manually through every level of the component tree.**

It provides a central place where data is stored, and any component can access it directly.

Example

Instead of passing a logged-in user from

```
App → Dashboard → Sidebar → Profile
```

using props,

Context API allows **Profile** to access the user directly.

● 2. Why do we need Context API?

Imagine an application where many components need the same data.

Examples:

- Logged-in User
- Theme (Dark/Light)
- Language
- Shopping Cart
- Authentication Status

Without Context API, we have to pass props through every intermediate component.

This creates unnecessary complexity.

Context API solves this problem.

3. How Context API Works

Context API has three main parts.

Step 1: Create Context

Create a context.

```
const UserContext = createContext();
```

This creates a storage box.

Step 2: Provider

Provider shares the data.

```
<UserContext value={data}>  
  
<Child Component />  
  
</UserContext>
```

Every component inside the Provider can access the data.

Step 3: useContext()

Components receive the data.

```
const user = useContext(UserContext);
```

Now the component can directly use the data.

4. Context API Flow

```
createContext()
    |
Provider stores data
    |
Child Components
    |
useContext()
    |
Receive Data
```

Example

Logged In User

```
UserContext.jsx
import { createContext } from "react";

export const UserContext = createContext();

App.jsx
import { UserContext } from "../context/UserContext";
import Profile from "../Profile";
```

```
function App(){  
  
  const user = "Kabir";  
  
  return(  
  
    <UserContext.Provider value={user}>  
  
      <Profile/>  
  
    </UserContext.Provider>  
  
  );  
  
}  
  
export default App;
```

Profile.jsx

```
import { useContext } from "react";  
import { UserContext } from "../context/UserContext";  
  
function Profile(){  
  
  const user = useContext(UserContext);  
  
  return(  
  
    <h1>Welcome {user}</h1>  
  
  );  
  
}
```

```
}
```

```
export default Profile;
```

Output

Welcome Kabir

Use Case

Authentication System

Suppose a user logs in.

We receive

```
{  
  name: "Kabir",  
  email: "abc@gmail.com",  
  isLoggedIn: true  
}
```

Many components need this.

Navbar

Dashboard

Profile

Orders

Settings

Instead of passing props,

Store the user inside Context.

Flow

Login



User Context



Provider



Navbar

Dashboard

Profile

Orders

Settings

Every component directly receives

User Data

Step-by-Step

```
Step 1
User logs in.
↓
Step 2
Store user inside Context.
↓
Step 3
Provider shares user.
↓
Step 4
Components call
useContext()

↓
Step 5
Display user information.
```

Benefits

- Single source of truth
 - Easy authentication
 - Less code
 - No repeated props
-

Advantages of Context API

- Built into React
 - No external library required
 - Eliminates props drilling
 - Easy to use
 - Cleaner code
 - Reusable
 - Good for global data
 - Improves maintainability
-

Disadvantages of Context API

- Not ideal for very complex state management
 - Frequent updates can re-render many components
 - Harder to optimize in large applications
 - Redux, Zustand, or similar libraries may be better for large-scale apps
-

Interview Questions

1. What is Context API?

Context API is a React feature that allows global data sharing between components without props drilling.

2. What is Props Drilling?

Passing data through multiple intermediate components using props, even if those components don't need the data.

3. Which hooks are used with Context API?

- `createContext()`

- `useContext()`

4. What is a Provider?

A Provider is a component that supplies data to all child components inside it.

5. Can Context API replace Redux?

For small and medium applications, often yes. For large applications with complex state management, Redux Toolkit or Zustand is usually more suitable.
